Interpolación polinomial

Lagrange, Newton, Hermite y aplicaciones

Oswaldo José Velásquez Castañón

Métodos Numéricos – 2026-2

Interpolación polinomial

Supongamos que tenemos $x_0, x_1, \ldots, x_n$, $(n+1)$ puntos distintos del eje real y $f_0, f_1, \ldots, f_n$ los valores correspondientes de la ordenada. Buscamos un polinomio de grado menor o igual a n

$$P_n(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

tal que $P_n(x_i) = f_i, \quad i = 0, 1, \ldots, n.$

Interpolación polinomial

Supongamos que tenemos $x_0, x_1, \ldots, x_n$, $(n+1)$ puntos distintos del eje real y $f_0, f_1, \ldots, f_n$ los valores correspondientes de la ordenada. Buscamos un polinomio de grado menor o igual a n

$$P_n(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

tal que $P_n(x_i) = f_i, \quad i = 0, 1, \ldots, n.$

Teorema

Para $(n+1)$ puntos $(x_0, f_0), (x_1, f_1), \ldots, (x_n, f_n) \in \mathbb{R}^2$ con abscisas distintas (esto es, $x_i \neq x_j$ para $i \neq j$), existe un único polinomio P_n de grado menor o igual a n que cumple las condiciones interpolantes:

$$P_n(x_i) = f_i, \quad i = 0, 1, \ldots, n$$

De hecho, las condiciones interpolantes

$$a_0 + a_1 x_i + a_2 x_i^2 + \cdots + a_n x_i^n = f_i, \quad i = 0, 1, \ldots, n$$

constituyen un sistema lineal de $n+1$ ecuaciones con $n+1$ incógnitas (los coeficientes del polinomio), que escribimos en la forma

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \ldots & x_0^n \\ 1 & x_1 & x_1^2 & \ldots & x_1^n \\ \vdots & & & & \vdots \\ 1 & x_n & x_n^2 & \ldots & x_n^n \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} f_0 \\ f_1 \\ \vdots \\ f_n \end{bmatrix}.$$

La matriz $(n+1) \times (n+1)$ del sistema es inversible, y por lo tanto el sistema posee solución única.

Consideramos los siguientes datos:

j	0	1	2	3
x_j	1	3	5	7
f_j	2.5	3.5	7.5	5.5

Consideramos los siguientes datos:

j	0	1	2	3
x_j	1	3	5	7
f_j	2.5	3.5	7.5	5.5

El único polinomio interpolante para esta tabla de valores tiene la forma

$$P_3(x) = a_0 + a_1 x + a_2 x^2 + a_3 x^3.$$

Consideramos los siguientes datos:

j	0	1	2	3
x_j	1	3	5	7
f_j	2.5	3.5	7.5	5.5

El único polinomio interpolante para esta tabla de valores tiene la forma

$$P_3(x) = a_0 + a_1 x + a_2 x^2 + a_3 x^3.$$

Los coeficientes a_0, a_1, a_2 y a_3 resuelven el sistema

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 3 & 9 & 27 \\ 1 & 5 & 25 & 125 \\ 1 & 7 & 49 & 343 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 2.5 \\ 3.5 \\ 7.5 \\ 5.5 \end{bmatrix}$$

Consideramos los siguientes datos:

j	0	1	2	3
x_j	1	3	5	7
f_j	2.5	3.5	7.5	5.5

El único polinomio interpolante para esta tabla de valores tiene la forma

$$P_3(x) = a_0 + a_1 x + a_2 x^2 + a_3 x^3.$$

Los coeficientes a_0, a_1, a_2 y a_3 resuelven el sistema

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 3 & 9 & 27 \\ 1 & 5 & 25 & 125 \\ 1 & 7 & 49 & 343 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 2.5 \\ 3.5 \\ 7.5 \\ 5.5 \end{bmatrix}$$

Resolviendo el sistema obtenemos $a_0 = 5.93750$, $a_1 = -5.31250$, $a_2 = 2.06250$ y $a_3 = -0.18750$, de donde

$$P_3(x) = 5.93750 - 5.31250x + 2.06250x^2 - 0.18750x^3.$$

Fórmula de Lagrange

Si (x_k, f_k), $k = 0, \ldots, n$ son los $n+1$ puntos antes considerados, el polinomio interpolante se puede obtener directamente por la fórmula

$$P_n(x) = f_0 L_0(x) + f_1 L_1(x) + \cdots + f_n L_n(x)$$

donde los polinomios $L_0, L_1, \ldots, L_n$ tienen grado n, si se exige que

$$L_j(x_k) = \begin{cases} 0, & j \neq k \\ 1, & j = k \end{cases}$$

Esto se cumple para los *polinomios básicos de Lagrange*

$$L_j(x) = \prod_{\substack{k=0 \\ k \neq j}}^{n} \frac{x - x_k}{x_j - x_k}.$$

Consideramos los datos de la tabla

j	0	1	2
x_j	0	0.5	1
f_j	1.1	0.9	0.6

Consideramos los datos de la tabla

j	0	1	2
x_j	0	0.5	1
f_j	1.1	0.9	0.6

Los polinomios básicos de Lagrange son

$$
\begin{aligned}
L_0(x) &= \frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)} = \frac{(x-0.5)(x-1)}{(0-0.5)(0-1)} = 2x^2-3x+1 \\
L_1(x) &= \frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)} = \frac{(x-0)(x-1)}{(0.5-0)(0.5-1)} = -4x^2+4x \\
L_2(x) &= \frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)} = \frac{(x-0)(x-0.5)}{(1-0)(1-0.5)} = 2x^2-x
\end{aligned}
$$

Consideramos los datos de la tabla

j	0	1	2
x_j	0	0.5	1
f_j	1.1	0.9	0.6

Los polinomios básicos de Lagrange son

$$\begin{aligned} L_0(x) &= \frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)} = \frac{(x-0.5)(x-1)}{(0-0.5)(0-1)} = 2x^2-3x+1 \\ L_1(x) &= \frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)} = \frac{(x-0)(x-1)}{(0.5-0)(0.5-1)} = -4x^2+4x \\ L_2(x) &= \frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)} = \frac{(x-0)(x-0.5)}{(1-0)(1-0.5)} = 2x^2-x \end{aligned}$$

Luego:

$$\begin{aligned} P_2(x) &= 1.1(2x^2-3x+1)+0.9(-4x^2+4x)+0.6(2x^2-x) \\ &= -0.2x^2-0.3x+1.1 \end{aligned}$$

Forma de Newton

Se inicia con el polinomio de grado 0 (constante) que pasa por el punto (x_0, f_0)

$$P_0(x) = f_0.$$

Añadiendo (x_1, f_1), se construye un polinomio $P_1(x)$ de grado no mayor que 1 tal que

$$P_1(x_0) = f_0, \quad P_1(x_1) = f_1.$$

Sea $P_1(x) = P_0(x) + q_1(x)$, donde $grado(q_1) \leq 1$. Como $P_1(x_0) = f_0$, entonces $q_1(x_0) = 0$, luego $x - x_0$ divide $q_1(x)$ y $q_1(x) = (x - x_0)\alpha(x)$. Pero el grado de q_1 es ≤ 1, luego $\alpha(x) = \alpha$ (constante). Como $P_1(x_1) = f_1$, entonces $f_1 = P_1(x_1) = f_0 + \alpha(x_1 - x_0)$. De esto último

$$\alpha = \frac{f_1 - f_0}{x_1 - x_0}$$

Denotando $f[x_0] = f_0$, $f[x_1] = f_1$ y $\alpha = f[x_0, x_1]$, se tiene que

$$f[x_0, x_1] = \frac{f[x_1] - f[x_0]}{x_1 - x_0}$$

y

$$P_1(x) = f[x_0] + f[x_0, x_1](x - x_0).$$

Ahora supongamos que tenemos $n+1$ puntos interpolantes (x_i, f_i) con $0 \leq i \leq n$, y el polinomio $P_{n-1}(x)$ de grado $\leq n-1$ que interpola los n primeros puntos, esto es

$$P_{n-1}(x_i) = f_i, \quad 0 \leq i \leq n-1.$$

De manera análoga a la anterior, se muestra que el polinomio $P_n(x)$ que interpola todos los puntos (incluido (x_n, f_n)) toma la forma

$$P_n(x) = P_{n-1}(x) + f[x_0, x_1, \ldots, x_n](x - x_0)(x - x_1)\cdots(x - x_{n-1}),$$

donde $f[x_0, x_1, \ldots, x_n]$ es el coeficiente de x^n en el desarrollo de $P_n(x)$. Reutilizando la recurrencia, el polinomio interpolante P_n puede ser escrito como

$$P_n(x) = f[x_0] + f[x_0, x_1](x - x_0) + \cdots + f[x_0, x_1, \ldots, x_n](x - x_0)\ldots(x - x_{n-1}).$$

Denotemos con $f[x_j, x_{j+1}, \ldots, x_{j+k}]$ al coeficiente de x^k en el polinomio interpolante que pasa por los puntos $(x_j, f_j), (x_{j+1}, f_{j+1}), \ldots, (x_{j+k}, f_{j+k})$. Se cumple:

$$f[x_0] = f_0, \quad f[x_1] = f_1, \quad \ldots, \quad f[x_n] = f_n$$

y se muestra que

$$f[x_j, \ldots, x_{j+k}] = \frac{f[x_{j+1}, \ldots, x_{j+k}] - f[x_j, \ldots, x_{j+k-1}]}{x_{j+k} - x_j}$$

Para calcular $f[x_0, \ldots, x_n]$, se utiliza la *tabla de diferencias divididas*:

x	$f[x]$	$f[x,x]$	$f[x,x,x]$	$f[x,x,x,x]$	$\ldots$
x_0	$f_0 := f[x_0]$				
		$f[x_0,x_1] = \frac{f[x_1]-f[x_0]}{x_1-x_0}$			
x_1	$f_1 := f[x_1]$		$f[x_0,x_1,x_2] = \frac{f[x_1,x_2]-f[x_0,x_1]}{x_2-x_0}$		
		$f[x_1,x_2] = \frac{f[x_2]-f[x_1]}{x_2-x_1}$		$f[x_0,x_1,x_2,x_3]$	
x_2	$f_2 := f[x_2]$		$f[x_1,x_2,x_3] = \frac{f[x_2,x_3]-f[x_1,x_2]}{x_3-x_1}$		$\ldots$
		$f[x_2,x_3] = \frac{f[x_3]-f[x_2]}{x_3-x_2}$		$f[x_1,x_2,x_3,x_4]$	
x_3	$f_3 := f[x_3]$		$f[x_2,x_3,x_4] = \frac{f[x_3,x_4]-f[x_2,x_3]}{x_4-x_2}$	$\vdots$	
		$f[x_3,x_4] = \frac{f[x_4]-f[x_3]}{x_4-x_3}$	$\vdots$		
x_4	$f_4 := f[x_4]$	$\vdots$			
$\vdots$	$\vdots$				
x_n	$f_n := f[x_n]$				

Retomamos el ejemplo de la tabla:

j	0	1	2	3
x_j	1	3	5	7
f_j	2.5	3.5	7.5	5.5

Retomamos el ejemplo de la tabla:

j	0	1	2	3
x_j	1	3	5	7
f_j	2.5	3.5	7.5	5.5

El esquema de diferencias divididas es

x	$f[x]$	$f[x,x]$	$f[x,x,x]$	$f[x,x,x,x]$
1	$f[1] = 2.5$			
		$f[1,3] = 0.5$		
3	$f[3] = 3.5$		$f[1,3,5] = 0.375$	
		$f[3,5] = 2$		$f[1,3,5,7] = -0.1875$
5	$f[5] = 7.5$		$f[3,5,7] = -0.75$	
		$f[5,7] = -1$		
7	$f[7] = 5.5$			

Retomamos el ejemplo de la tabla:

j	0	1	2	3
x_j	1	3	5	7
f_j	2.5	3.5	7.5	5.5

El esquema de diferencias divididas es

x	$f[x]$	$f[x,x]$	$f[x,x,x]$	$f[x,x,x,x]$
1	$f[1] = 2.5$			
		$f[1,3] = 0.5$		
3	$f[3] = 3.5$		$f[1,3,5] = 0.375$	
		$f[3,5] = 2$		$f[1,3,5,7] = -0.1875$
5	$f[5] = 7.5$		$f[3,5,7] = -0.75$	
		$f[5,7] = -1$		
7	$f[7] = 5.5$			

El polinomio interpolante es

$$\begin{aligned} P_3(x) &= f[1] + f[1,3](x-1) + f[1,3,5](x-1)(x-3) \\ &\quad + f[1,3,5,7](x-1)(x-3)(x-5) \\ &= 2.5 + 0.5(x-1) + 0.375(x-1)(x-3) - 0.1875(x-1)(x-3)(x-5) \end{aligned}$$

Error de la aproximación polinomial

Suponemos ahora que $f : \mathbb{R} \to \mathbb{R}$ es una función dada, y el polinomio interpolante $P_n(x)$ la interpola en $n+1$ puntos dados. Puesto que, en general $f(x) \neq P_n(x)$, hay un error en la aproximación de $f(x)$ mediante $P_n(x)$. Como

$$f(x) = P_n(x) + f[x_0, x_1, \ldots, x_n, x] \prod_{i=0}^{n} (x - x_i),$$

el error en la aproximación por el polinomio $P_n(x)$ (y que pasa por los $n+1$ puntos prefijados) está dado por

$$E(x) = f[x_0, x_1, \ldots, x_n, x] \prod_{i=0}^{n} (x - x_i).$$

Podemos calcular este error empleando un punto adicional x en el esquema de diferencias divididas, si conocemos $f(x)$.

Teorema del error de interpolación

Teorema

Sea $f \in C^{n+1}[a,b]$, con nodos distintos $x_0, \ldots, x_n \in [a,b]$. Sea P_n el polinomio de grado $\leq n$ que satisface $P_n(x_i) = f(x_i)$, $i = 0, \ldots, n$. Para cada $x \in [a,b]$ que no sea un nodo, existe

$$\xi \in \Big(\text{mín}\{x, x_0, \ldots, x_n\}, \text{máx}\{x, x_0, \ldots, x_n\}\Big)$$

tal que

$$f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^{n} (x - x_i).$$

En los nodos, el error es cero.

Cota del error de interpolación

Si $M_{n+1} = \max_{t\in[a,b]} |f^{(n+1)}(t)|$, entonces

$$|f(x) - P_n(x)| \leq \frac{M_{n+1}}{(n+1)!} \left| \prod_{i=0}^{n} (x - x_i) \right|.$$

La cota depende tanto de la función como de la distribución de nodos.

Error de aproximación y error de cálculo

La fórmula supone que evaluamos el interpolante exacto. El cálculo en punto flotante agrega errores de redondeo.

El residual en los nodos verifica las condiciones interpolantes, pero no mide el error entre los nodos.

Nota sobre interpolación con condiciones sobre las derivadas:
Observe que

$$\lim_{h\to 0} \frac{f[x+h]-f[x]}{h} = f'(x),$$

por lo que definimos

$$f[x,x] = f'(x).$$

Igualmente

$$f[x,x,x] = \frac{1}{2}f''(x), \quad f[x,x,x,x] = \frac{1}{6}f'''(x)$$

y, en general (para $n+1$ puntos iguales x)

$$f[x,x,\ldots,x] = \frac{1}{n!}f^{(n)}(x).$$

Esto nos permite reemplazar condiciones sobre las derivadas en la tabla de diferencias divididas, siempre y cuando el número de condiciones totales sea igual al número de coeficientes a calcular. La interpolación con este tipo de condiciones se denomina *interpolación de Hermite*.

Consideremos la tabla

x_i	x_0	x_1
f_i	f_0	f_1
f_i'	f_0'	f_1'
f_i''	f_0''	f_1''

Tenemos 6 condiciones interpolantes. Se está buscando un polinomio de grado ≤ 5: $P_5(x)$.

x	$f[x]$	$f[x,x]$	$f[x,x,x]$	$f[x,x,x,x]$	$f[x,x,x,x,x]$	$f[x,x,x,x,x,x]$
x_0	$f[x_0]$					
		$f'(x_0)$				
x_0	$f[x_0]$		$\frac{1}{2}f''(x_0)$			
		$f'(x_0)$		$f[x_0,x_0,x_0,x_1]$		
x_0	$f[x_0]$		$f[x_0,x_0,x_1]$		$f[x_0,x_0,x_0,x_1,x_1]$	
		$f[x_0,x_1]$		$f[x_0,x_0,x_1,x_1]$		$f[x_0,x_0,x_0,x_1,x_1,x_1]$
x_1	$f[x_1]$		$f[x_0,x_1,x_1]$		$f[x_0,x_0,x_1,x_1,x_1]$	
		$f'(x_1)$		$f[x_0,x_1,x_1,x_1]$		
x_1	$f[x_1]$		$\frac{1}{2}f''(x_1)$			
		$f'(x_1)$				
x_1	$f[x_1]$					

Entonces

$$\begin{aligned} P_5(x) =& b_0 + b_1(x-x_0) + b_2(x-x_0)^2 + b_3(x-x_0)^3 \\ &+ b_4(x-x_0)^3(x-x_1) + b_5(x-x_0)^3(x-x_1)^2 \end{aligned}$$

donde los $b_i, \quad i = 0, \ldots, 5$ son los valores de la primera línea oblicua.

Spline cúbico

Sean los puntos (x_i, f_i), $i = 1, \ldots, n$ con

$$a = x_1 < x_2 < \cdots < x_{n-1} < x_n = b.$$

Un *spline cúbico* es una función $S : [a, b] \to \mathbb{R}$ tal que

1. $S(x)$ es un polinomio de grado ≤ 3 en cada intervalo $[x_j, x_{j+1}]$, $j = 1, 2, \ldots, n-1$;
2. $S(x_i) = f_i$, $i = 1, 2, \ldots, n$;
3. S, S' y S'' son funciones continuas en $[a, b]$ (el spline S es dos veces continuamente diferenciable en $[a, b]$).

Tipos de splines

Un spline cúbico es

1. *natural* si
$$S''(a) = S''(b) = 0,$$
2. *anclado* si
$$S'(a) = \alpha, \quad S'(b) = \beta$$
para ciertos valores α, β.
3. *periódico* si S es una función periódica de periodo $b - a$, esto es
$$S(a) = S(b), \quad S'(a) = S'(b), \quad S''(a) = S''(b).$$

Determine si la siguiente función es un spline cúbico:

$$f(x) = \begin{cases} x^3 + x, & x \in [-1, 0] \\ x^3 - x, & x \in [0, 1]. \end{cases}$$

Determine todos los valores de a, b, c, d y e para los cuales la siguiente función es un spline cúbico:

$$f(x) = \begin{cases} a(x-2)^2 + b(x-1)^3, & x \in [-10, 1] \\ c(x-2)^2, & x \in [1, 3] \\ d(x-2)^2 + e(x-3)^3, & x \in [3, 10]. \end{cases}$$

Construcción del spline cúbico

Observe que si un spline S tiene n nodos, estará definido por $n-1$ polinomios cúbicos, de donde se necesitan $4(n-1)$ coeficientes.
Definimos $h_i = x_{i+1} - x_i$ para $i = 1, 2, \ldots, n-1$. En cada intervalo $[x_i, x_{i+1}]$, desarrollamos el esquema de diferencias divididas siguiente:

x_i	f_i			
		$\lambda_i = ?$		
x_i	f_i		$\frac{m_i - \lambda_i}{h_i}$	
		m_i		$\frac{\lambda_{i+1} - 2m_i + \lambda_i}{h_i^2}$
x_{i+1}	f_{i+1}		$\frac{\lambda_{i+1} - m_i}{h_i}$	
		λ_{i+1}		
x_{i+1}	f_{i+1}			

El polinomio cúbico que representa el spline en $[x_i, x_{i+1}]$ está dado por

$$S(x) = f_i + \lambda_i(x - x_i) + \frac{m_i - \lambda_i}{h_i}(x - x_i)^2 + \frac{\lambda_{i+1} - 2m_i + \lambda_i}{h_i^2}(x - x_i)^2(x - x_{i+1}).$$

Para un spline natural, con $m_i = (f_{i+1} - f_i)/h_i$ y $\lambda_i = S'(x_i)$, debemos resolver el sistema lineal

$$
\begin{aligned}
&2\lambda_1 + \lambda_2 = 3m_1 \\
&\tfrac{h_i}{h_{i-1}}\lambda_{i-1} + 2\Big(1 + \tfrac{h_i}{h_{i-1}}\Big)\lambda_i + \lambda_{i+1} = 3\Big(m_i + \tfrac{h_i}{h_{i-1}}m_{i-1}\Big), \quad i = 2, \ldots, n-1 \\
&\lambda_{n-1} + 2\lambda_n = 3m_{n-1}
\end{aligned}
$$

Diferenciación numérica

Consideramos una función f interpolada por un polinomio $P_n(x)$ en $n+1$ puntos. Sean $(x_i, f_i), \quad i = 0, 1, \ldots, n$ puntos interpolantes. De la fórmula de Lagrange:

$$P_n(x) = \sum_{j=0}^{n} f_j \alpha_j \prod_{\substack{k=0 \\ k \neq j}}^{n} (x - x_k)$$

con $\alpha_j = \prod_{\substack{k=0 \\ k \neq j}}^{n} \frac{1}{x_j - x_k}$.

Se obtiene:

$$\frac{d^n P_n}{dx^n}(x) = \sum_{j=0}^{n} f_j \alpha_j n! \sim f^{(n)}(x)$$

que resulta una aproximación de la n-ésima derivada $f^{(n)}$.

Fórmulas de diferencias para las derivadas:

Consideramos puntos igualmente espaciados, con $x_{j+1} - x_j = h$. Entonces $\alpha_j = \frac{(-1)^{n-j}}{h^n n!}\binom{n}{j}$, y obtenemos la fórmula

$$f^{(n)}(x_0) \approx \sum_{j=0}^{n} f_j \frac{(-1)^{n-j}}{h^n}\binom{n}{j}.$$

Para $n = 1, 2, 3$, con $f_j = f(x + jh)$:

$$f'(x) \sim \tfrac{f_1 - f_0}{h} = \tfrac{f(x+h) - f(x)}{h}$$

$$f''(x) \sim \tfrac{f_2 - 2f_1 + f_0}{h^2} = \tfrac{f(x+2h) - 2f(x+h) + f(x)}{h^2}$$

$$f'''(x) \sim \tfrac{f_3 - 3f_2 + 3f_1 - f_0}{h^3} = \tfrac{f(x+3h) - 3f(x+2h) + 3f(x+h) - f(x)}{h^3}$$

Interpolación en dimensión dos

Dado un rectángulo $[a, b] \times [c, d]$, consideremos una malla regular, formada por puntos igualmente espaciados en cada dimensión

$$x_i = a + ih, i = 0, \ldots, n, \quad y_j = c + jk, j = 0, \cdots, m$$

con $h = \frac{b-a}{n}$, $k = \frac{d-c}{m}$.
Denotamos

$$p_{i,j} = (x_i, y_j), \quad f_{i,j} = f(p_{i,j});$$

consideramos entonces el problema de interpolación en cada rectángulo

$$R_{i,j} = [x_i, x_{i+1}] \times [y_j, y_{j+1}], \quad i = 0, \ldots, n-1, \ j = 0, \ldots, m-1.$$

La función

$$\begin{aligned} P_{i,j}(x,y) =& f_{i,j} + mx_{i,j}(x - x_i) + my_{i,j}(y - y_j) \\ &+ \frac{mx_{i,j+1} - mx_{i,j}}{k}(x - x_i)(y - y_j), \end{aligned}$$

para $(x, y) \in R_{ij}$, donde

$$mx_{i,j} = \frac{f_{i+1,j} - f_{i,j}}{h}, \quad my_{i,j} = \frac{f_{i,j+1} - f_{i,j}}{k}$$

cumple las condiciones interpolantes en el rectángulo $R_{i,j}$.
Con las condiciones establecidas, la función $P : [a, b] \times [c, d] \to \mathbb{R}$ tal que

$$P(x,y) = P_{i,j}(x,y), \text{ para } (x,y) \in R_{i,j}, \quad i = 0, \ldots, n-1, \ j = 0, \ldots, m-1$$

cumple las condiciones interpolantes, y es una función continua (en general, no diferenciable en las interfaces).

En cada rectángulo $R_{i,j}$, la función $P_{i,j}$ también puede escribirse en forma similar a la forma de Lagrange

$$P(x,y) = f_{i,j}\alpha_{i,j}(x,y) + f_{i+1,j}\beta_{i,j}(x,y) + f_{i+1,j+1}\gamma_{i,j}(x,y) + f_{i,j+1}\delta_{i,j}(x,y),$$

donde

$$\alpha_{i,j}(x,y) = \Big(1 - \frac{x-x_i}{h}\Big)\Big(1 - \frac{y-y_j}{k}\Big),$$
$$\beta_{i,j}(x,y) = \Big(\frac{x-x_i}{h}\Big)\Big(1 - \frac{y-y_j}{k}\Big),$$
$$\gamma_{i,j}(x,y) = \Big(\frac{x-x_i}{h}\Big)\Big(\frac{y-y_j}{k}\Big),$$
$$\delta_{i,j}(x,y) = \Big(1 - \frac{x-x_i}{h}\Big)\Big(\frac{y-y_j}{k}\Big).$$

Interpolación en una malla triangulada

Consideramos una malla triangular de puntos $p_i = (x_i, y_i)$, $i = 1, \ldots, n$. Dados tres puntos p_i, p_j, p_k no colineales formando un triángulo, formamos una función interpolante P sobre el triángulo $\Delta = p_i p_j p_k$

$$P(x, y) = f_i \phi_i(x, y) + f_j \phi_j(x, y) + f_k \phi_k(x, y),$$

donde

$$\phi_i(x, y) = F_{j,k}(x, y), \quad \phi_j(x, y) = F_{k,i}(x, y), \quad \phi_k(x, y) = F_{i,j}(x, y),$$

donde

$$F_{s,t} = \frac{1}{A_{i,j,k}}(\alpha_{s,t} + \beta_{s,t} x - \gamma_{s,t} y),$$

con

$$\alpha_{s,t} = x_s y_t - x_t y_s, \quad \beta_{s,t} = y_s - y_t, \quad \gamma_{s,t} = x_s - x_t,$$

y

$$A_{i,j,k} = \begin{vmatrix} 1 & x_i & y_i \\ 1 & x_j & y_j \\ 1 & x_k & y_k \end{vmatrix}.$$

Observación. De hecho, la gráfica de P es, dentro del triángulo $\Delta = p_i p_j p_k$, el plano que pasa por los puntos (x_i, y_i, f_i), (x_j, y_j, f_j) y (x_k, y_k, f_k). Podemos determinar la ecuación del plano haciendo

$$P(x, y) = a + bx + cy,$$

y luego estableciendo las ecuaciones (condiciones interpolantes)

$$P(x_s, y_s) = a + bx_s + cy_s = f_s, \quad s = i, j, k$$

esto es

$$\begin{bmatrix} 1 & x_i & y_i \\ 1 & x_j & y_j \\ 1 & x_k & y_k \end{bmatrix} \begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} f_i \\ f_j \\ f_k \end{bmatrix}$$

para determinar a, b, c. Las fórmulas anteriores se obtienen aplicando la regla de Cramer para resolver este sistema.

Vandermonde en NumPy

Retomamos los mismos cuatro puntos de la presentación.

```
import numpy as np
from numpy.polynomial.polynomial import polyval

x = np.array([1., 3., 5., 7.])
f = np.array([2.5, 3.5, 7.5, 5.5])
V = np.vander(x, increasing=True)
a = np.linalg.solve(V, f)
print(a)
# [5.9375, -5.3125, 2.0625, -0.1875]
print(polyval(4., a))  # 5.6875
```

`increasing=True` ordena las columnas como $1, x, x^2, x^3$. `polyval` usa los coeficientes en ese mismo orden.

Diferencias divididas en NumPy

La recurrencia anterior permite obtener los coeficientes de Newton.

```
def diferencias_divididas(x, f):
    x = np.asarray(x, dtype=float)
    c = np.array(f, dtype=float, copy=True)
    if (x.ndim != 1 or c.ndim != 1
            or len(x) != len(c) or len(x) == 0):
        raise ValueError("Datos incompatibles")
    if len(np.unique(x)) != len(x):
        raise ValueError("Nodos repetidos")
    for k in range(1, len(x)):
        c[k:] = ((c[k:] - c[k-1:-1])
                 / (x[k:] - x[:-k]))
    return c
```

La función requiere nodos distintos. Hermite necesita un tratamiento específico de los nodos repetidos.

Evaluación anidada de Newton

Para cuatro nodos,

$$P_3(t) = c_0 + (t - x_0)\Big[c_1 + (t - x_1)[c_2 + (t - x_2)c_3]\Big].$$

```
def evaluar_newton(t, x, c):
    t = np.asarray(t, dtype=float)
    p = np.full_like(t, c[-1], dtype=float)
    for k in range(len(c)-2, -1, -1):
        p = c[k] + (t-x[k])*p
    return p

c = diferencias_divididas(x, f)
print(c)  # [2.5, 0.5, 0.375, -0.1875]
print(evaluar_newton(4., x, c))  # 5.6875
```

Construcción: $O(n^2)$. Evaluación por punto: $O(n)$.

Verificación de la implementación

```
q = np.linspace(x.min(), x.max(), 101)
p_v = polyval(q, a)
p_n = evaluar_newton(q, x, c)

assert np.allclose(polyval(x, a), f)
assert np.allclose(evaluar_newton(x, x, c), f)
assert np.allclose(p_v, p_n)

residual = np.max(np.abs(polyval(x, a)-f))
condicion = np.linalg.cond(V)
print(residual, condicion)
```

Vandermonde y Newton representan el mismo polinomio en aritmética exacta.
Una matriz de Vandermonde mal condicionada puede amplificar perturbaciones.
La coincidencia de los valores en los nodos no garantiza una buena aproximación de la función en todo el intervalo.

Convenciones de NumPy

- `numpy.polynomial.polynomial.polyval(t, a)` recibe $a_0, a_1, \ldots, a_n$ en orden creciente de potencias.
- `numpy.polyval(a, t)` recibe los coeficientes en orden decreciente.
- `numpy.interp` realiza interpolación lineal por tramos.
- Un ajuste de grado menor que el número de nodos menos uno resuelve un problema diferente de la interpolación exacta.

Documentación oficial de NumPy: vander, linalg.solve, polynomial.polyval.